علیرضا کریمی ۸۱۰۱۰۱۴۹۲ DB_HW6

۱)

الف) clustered tree index: کلا برای بازه‌ها بهتر است از tree index استفاده کنیم. حالا clustered بودن آن به نوعی باعث sorted بودن آن شده و از هزینه آوردن page‌های زیاد به main memory جلوگیری می‌کند هزینه ⟵ $\left(\log_F^{\overset{\text{1.5 برابر}}{KB}} + \#\text{match pages}\right) \times D$

فرض: هر page $\frac{2}{3}$ پر باشد

ب) heap file: درصورتی که ترتیب اهمیتی ندارد، پس لازم به ساختار درختی برای درج نیست و به راحتی می‌توان از همان heap استفاده کرد. هزینه درج، 2D است (یعنی $\underbrace{\text{خواندن آن Page}}_{D}$، update کردن و $\underbrace{\text{نوشتن آن}}_{D}$) برای اسکن هم که در همه موارد حداقل باید تمام موارد رو چک کنیم که به ترتیبی هم نیاز ندارد پس همان B x D کافی است (B تعداد page‌ها است)

ج) کلا hash-based: روش‌های hash-based چون از تابع hash استفاده می‌کند، در D قابل تعیین equality search هستند.

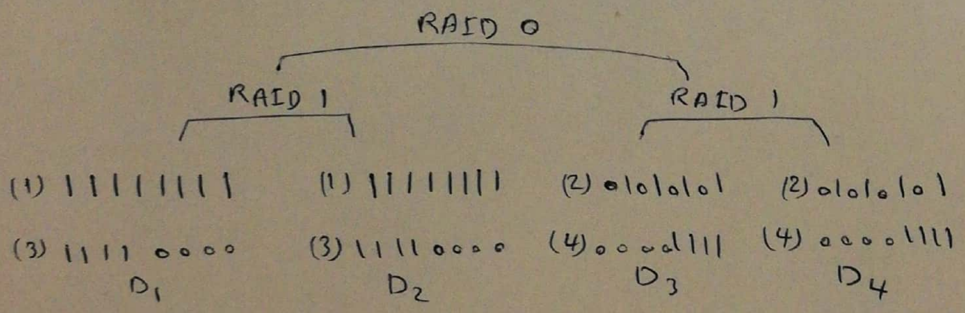
۲) کلا از unclustered‌ها زمانی استفاده می‌کنیم که ویژگی ما به نوعی unique و selective باشد. برای حالتی که مثلا تعداد زیادی کارمند از یک نوع sal با sal<20 داشته باشیم، بمراتب clustered بهتر از unclustered است. چون فرض کن هرکدوم از 250 تا کارمند با sal=10 (یا همان sal<20) در page‌های جداگانه باشند. یعنی باید 250 تا page بیاد توی main memory. درصورتی که اگر clustered بود، شاید هم record‌ها در یک page بیشتر بود. هزینه IO زیاد می‌شود.

۳)

RAID لول ۱+۰:

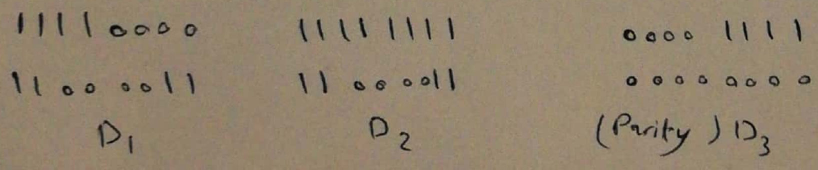
(D2 و D4 دیسک‌های پشتیبان)

(D1 و D3 دیسک‌های اصلی)


RAID 0
RAID 1
RAID 1
(1) 11111111
(1) 11111111
(2) 01010101
(2) 01010101
(3) 11110000
(3) 11110000
(4) 00001111
(4) 00001111
D1
D2
D3
D4

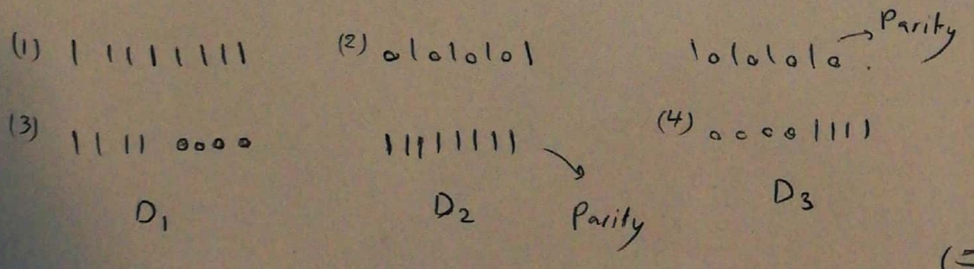
RAID لول ۳:

(D1 و D2 دیسک‌های اصلی و D3 دیسک Parity)


11110000
11000011
D1
11111111
11000011
D2
00001111
00000000
(Parity) D3

RAID لول 5:

(D1 و D2 و D3 دیسک‌های اصلی و به نوعی کمکی برای Parity است و Parity در بین دیسک‌ها پخش شده است)


(1) 11111111
(3) 11110000
D1
(2) 01010101
11111111
D2
Parity
10101010
Parity
(4) 00001111
D3

(۴

	LRU state	LRU	MRU state	MRU
۱	M	۱	M	۱
۲	M	۱ ۲	M	۱ ۲
۳	M	۱ ۲ ۳	M	۱ ۲ ۳
۴	M	۱ ۲ ۳ ۴	M	۱ ۲ ۳ ۴
۵	M	۱ ۲ ۳ ۴ ۵	M	۱ ۲ ۳ ۴ ۵
۲	H	۱ ۲ ۳ ۴ ۵	H	۱ ۲ ۳ ۴ ۵
۶	M	۶ ۲ ۳ ۴ ۵	M	۱ ۶ ۳ ۴ ۵
۲	H	۶ ۲ ۳ ۴ ۵	M	۱ ۲ ۳ ۴ ۵
۱	M	۶ ۲ ۱ ۴ ۵	H	۱ ۲ ۳ ۴ ۵
۴	H	۶ ۲ ۱ ۴ ۵	H	۱ ۲ ۳ ۴ ۵
۳	M	۶ ۲ ۱ ۴ ۳	H	۱ ۲ ۳ ۴ ۵
۶	H	۶ ۲ ۱ ۴ ۳	M	۱ ۲ ۶ ۴ ۵
۳	H	۶ ۲ ۱ ۴ ۳	M	۱ ۲ ۳ ۴ ۵

→ LRU hit rate = $\frac{5}{13}$

→ MRU hit rate = $\frac{4}{13}$

۵) برای clustered hash داریم:

هزینه equality: 2D (یک D برای پیدا کردن bucket و یکی دیگه برای پیدا کردن record از روی rid)

هزینه range: ابتدا باید بگوییم که حضور شاخص اینجا کمکی نمی‌کند و باید تمام page ها را بخوانیم ← B × D ← تعداد صفحات

برای clustered tree index داریم: چون که $\frac{۳}{۵}$ هر صفحه پر است و از طرفی data entry ها $\frac{1}{۵}$ سایز record هستند ← $\log_F \frac{۵}{3}\times\frac{1}{۵}B$

هزینه equality: اول با $\log_F \frac{B}{3}$ به data entry مورد نظر می‌رسیم و از روی آن، record مورد نظر را پیدا و می‌خوانیم.

$\longrightarrow (\log_F \frac{B}{3}+1)\times D \longrightarrow$ هزینه خواندن

هزینه range: اول با خواندن $\log_F \frac{B}{3}$ صفحه، data entry را پیدا می‌کنیم و بعد به page مورد نظر آن رفته و تا وقتی match می‌شود جلو می‌رویم و چون clustered است، record ها با درصد خوبی در کنار هم هستند و می‌توان فرض کرد با شانس خوب تعدادی از match record ها در یک صفحه هستند پس داریم:

$\longrightarrow \left(\log_F \frac{B}{3} + \text{Match pages} + \underbrace{\text{Match records}}\right)\times D$

(چون به ازای هر data record در بدترین حالت یک page می‌خوانیم اما اینجا cluster است ← برای cluster تا حدی optional و اضافه

ادامه 5) حتی اگر همه آن match record ها در یک page باشند، می‌توانیم از عبارت match record در فرمول صرف‌نظر کنیم.

۶)

۱. < entry-year , course count, grade >

چون ابتدا براساس entry year مرتب و تقسیم‌بندی کردیم و بعد براساس course count مرتب کنیم و بین اونهایی که course count آنها بالاتر از 4 است، کمترین grade را پیدا می‌کنیم.

۲. < entry year, grade >

ابتدا براساس entry year تقسیم‌بندی می‌کنیم و بعد grade های موجود را میانگین می‌گیریم.

۷)

الف) چون دوتا field متغیر داریم، به ازای هرکدام، pointer به انتهای آن field داریم. پس دو پوینتر و در مجموع ۲×۲=۴B (۴ بایت)

ب) ابتدا (حداکثر) طول ممکن را برای هر record حساب می‌کنیم:

4(for pid) + 4(for production_date) + 10 (for name) + 20 (for factory) +

اصلاح بخش الف

4(for record header) = 42 bytes

از طرفی به ازای هر record، $\overbrace{\text{طول ان (int)}}^{2B}$ $\overbrace{\text{و پوینتر به ابتدای}}^{4B}$ آن record در بخش footer قرار می‌گیرد. از طرفی در آن

به صورت کلی، یک پوینتر به ابتدای free space (2B) و یک counter برای تعداد slot ها در slot directory (4B)

پس در مجموع تعداد کل record هایی که می‌توانند در 4KB قرار بگیرند برابرست با:

(به‌گونه‌ای این تعداد حداقل تعداد record های ممکن است)

$$\left\lfloor \frac{4\times 1024-(2+4)}{42+2+4} \right\rfloor = \left\lfloor \frac{4092}{48} \right\rfloor = \text{۸۵}$$

البته اگر برای هر record برای متغیرهای متغیر، null فرض کنیم در ابتدا و آن ۲۰+۱۰ بایت برای field های متغیر را در نظر نگیریم

پس داریم: (به‌گونه‌ای حداکثر تعداد record ممکن)

$$\left\lfloor \frac{4092}{12+2+4} \right\rfloor = 227$$